

**MINISTRY OF EDUCATION AND TRAINING  
FPT UNIVERSITY**

**A TWO-TIER COGNITIVE ARCHITECTURE FOR  
AUTOMATED THREAT DETECTION AND RESPONSE  
USING AGENTIC AI**

by

Nguyen Duc Binh

A thesis submitted in conformity with the requirements  
for the degree of Master of Software Engineering

**MINISTRY OF EDUCATION AND TRAINING  
FPT UNIVERSITY**

**A TWO-TIER COGNITIVE ARCHITECTURE FOR  
AUTOMATED THREAT DETECTION AND RESPONSE  
USING AGENTIC AI**

by

Nguyen Duc Binh

A thesis submitted in conformity with the requirements  
for the degree of Master of Software Engineering

Supervisor:  
Dr. Bui Van Hieu  
Dr. Dang Van Hieu

## Abstract

In the contemporary cybersecurity landscape, Security Operations Centers (SOCs) are consistently overwhelmed by the sheer volume of network alerts, leading to severe alert fatigue and the potential oversight of critical Advanced Persistent Threats (APTs). To address the inherent limitations of traditional, static rule-based systems and the latency bottlenecks of processing raw logs directly through Large Language Models (LLMs), this thesis introduces SENTINEL: A Two-Tier Cognitive Architecture for Automated Threat Detection and Response. The proposed framework operates by deploying Welford's online statistical algorithm at the Transport Tier (Tier-1) to filter benign traffic at wire speed with minimal computational overhead. Subsequently, the Cognitive Agent Tier (Tier-2) leverages a LangGraph-driven AI Agent equipped with a Dual Retrieval-Augmented Generation (Dual-RAG) mechanism—integrating MITRE ATT&CK and NIST guidelines—to perform deep, contextual threat reasoning and mitigation.

Evaluated on the CSE-CIC-IDS2018 and DAPT2020 datasets, SENTINEL demonstrates substantial improvements in both detection accuracy and processing efficiency. Furthermore, the system incorporates robust AI security guardrails, including Delimited Data Encapsulation and HMAC log chaining, achieving near-perfect mitigation against known adversarial AI manipulation attacks and ensuring forensic data integrity. The findings underscore the efficacy of combining deterministic statistical filters with explainable Agentic AI, offering a highly scalable and secure automation solution for modern network defense.

## Acknowledgments

I would like to express my deepest and most sincere gratitude to my supervisors, Dr. Bui Van Hieu and Dr. Dang Van Hieu. Their precise academic direction, sharp professional feedback, and continuous guidance throughout this research have served as the cornerstone for the successful completion of this thesis.

I am also profoundly grateful to all the faculty members and staff at FPT University, particularly the Master of Software Engineering (MSE) program, for equipping me with invaluable academic knowledge, fostering a highly professional and modern research environment, and providing excellent resources during my master's studies.

Furthermore, I would like to extend my appreciation to the management and my colleagues at FPT Software for their support, understanding, and valuable professional discussions, which greatly helped me balance my professional duties and academic pursuits.

Finally and most importantly, I dedicate a special acknowledgment to my loving family, especially my wife and young child. Their silent sacrifices, patience, understanding, and unwavering moral support during my intensive experimental runs provided the ultimate strength and motivation for me throughout this master's journey.

# List of Figures

|           |                                                                                                                                                                                              |    |
|-----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| Figure 1. | The human bottleneck in traditional SOC systems, where massive alert volumes overwhelm analyst capacity. . . . .                                                                             | 15 |
| Figure 2. | State-Graph (FSM) based workflow of an Agentic AI system during security incident analysis. . . . .                                                                                          | 17 |
| Figure 3. | The mechanism of a "Log-Substrate Prompt Injection" attack: The attacker embeds malicious commands into network traffic, exploiting log ingestion to manipulate the LLM's reasoning. . . . . | 19 |

# List of Tables

|          |                                                                                        |    |
|----------|----------------------------------------------------------------------------------------|----|
| Table 2. | Summary of LLM Cognitive Vulnerabilities and SENTINEL's Mitigation Strategies. . . . . | 20 |
|----------|----------------------------------------------------------------------------------------|----|

## List of Abbreviations

| Abbreviation | Full Description                           |
|--------------|--------------------------------------------|
| AI           | Artificial Intelligence                    |
| APT          | Advanced Persistent Threat                 |
| C2           | Command and Control                        |
| CIDR         | Classless Inter-Domain Routing             |
| CNN          | Convolutional Neural Network               |
| CVE          | Common Vulnerabilities and Exposures       |
| CTI          | Cyber Threat Intelligence                  |
| DAPT         | Dynamic/Delayed Advanced Persistent Threat |
| DDoS         | Distributed Denial of Service              |
| DL           | Deep Learning                              |
| DoS          | Denial of Service                          |
| EDR          | Endpoint Detection and Response            |
| FSM          | Finite State Machine                       |
| GenAI        | Generative Artificial Intelligence         |
| HITL         | Human-in-the-Loop                          |
| HMAC         | Hash-based Message Authentication Code     |
| IDS          | Intrusion Detection System                 |
| IPS          | Intrusion Prevention System                |
| LLM          | Large Language Model                       |
| LSTM         | Long Short-Term Memory                     |
| MAS          | Multi-Agent System                         |
| ML           | Machine Learning                           |
| NGFW         | Next-Generation Firewall                   |
| RAG          | Retrieval-Augmented Generation             |
| RBA          | Runbook Automation                         |
| RRF          | Reciprocal Rank Fusion                     |
| SCA          | Software Composition Analysis              |
| SIEM         | Security Information and Event Management  |
| SOC          | Security Operations Center                 |
| SVM          | Support Vector Machine                     |
| TDIR         | Threat Detection and Incident Response     |
| TTL          | Time to Live                               |
| VRAM         | Video Random Access Memory                 |
| WAF          | Web Application Firewall                   |

# Contents

|                                                                                                |           |
|------------------------------------------------------------------------------------------------|-----------|
| <b>Abstract</b>                                                                                | <b>1</b>  |
| <b>Acknowledgments</b>                                                                         | <b>2</b>  |
| <b>List of Abbreviations</b>                                                                   | <b>5</b>  |
| <b>1 Introduction</b>                                                                          | <b>8</b>  |
| 1.1 Background and Motivation . . . . .                                                        | 8         |
| 1.1.1 The Evolution of Cybersecurity Threat Landscapes . . . . .                               | 8         |
| 1.1.2 The Role and Complexity of Modern SOCs . . . . .                                         | 8         |
| 1.1.3 The Alert Fatigue Crisis . . . . .                                                       | 9         |
| 1.1.4 The Inefficacy of Deterministic Rule-Based Systems . . . . .                             | 9         |
| 1.1.5 Practical Problems in Temporal Data Reduction . . . . .                                  | 9         |
| 1.1.6 The Integration Paradox of Generative AI . . . . .                                       | 9         |
| 1.2 Research Objectives . . . . .                                                              | 10        |
| 1.3 Research Questions . . . . .                                                               | 10        |
| 1.4 Scope and Object of Study . . . . .                                                        | 11        |
| 1.5 Research Methodology and Data Overview . . . . .                                           | 11        |
| 1.5.1 General and Specific Reasoning Paradigms . . . . .                                       | 11        |
| 1.5.2 Research Methodology & Execution Steps . . . . .                                         | 12        |
| 1.5.3 Empirical Data Characteristics . . . . .                                                 | 12        |
| 1.6 Related Works . . . . .                                                                    | 12        |
| 1.7 Contributions of the Thesis . . . . .                                                      | 13        |
| 1.8 Structure of the Thesis . . . . .                                                          | 13        |
| 1.9 Chapter Summary . . . . .                                                                  | 13        |
| <b>2 Theoretical Background</b>                                                                | <b>14</b> |
| 2.1 Overview of Security Operations Centers (SOCs) and the Cognitive Bot-<br>tleneck . . . . . | 14        |
| 2.2 Large Language Models (LLMs) and the Mathematics of Quantization . .                       | 15        |
| 2.3 Agentic AI Architecture and State-Graph Models . . . . .                                   | 16        |
| 2.4 Retrieval-Augmented Generation (RAG) and Hybrid Search Optimization                        | 17        |
| 2.5 LLM Cognitive Vulnerabilities and Prompt Injection Vectors . . . . .                       | 18        |
| 2.6 Welford's Online Statistical Algorithm . . . . .                                           | 20        |
| 2.7 Chapter Summary . . . . .                                                                  | 21        |
| <b>3 The Proposed SENTINEL Architecture</b>                                                    | <b>22</b> |
| 3.1 Overview of the Two-Tier Cognitive Architecture . . . . .                                  | 22        |
| 3.2 Tier-1: Real-Time Traffic Filtration Engine (RuleEngine & Welford) . . .                   | 22        |
| 3.3 Tier-2: LangGraph Cognitive AI Agent . . . . .                                             | 23        |

|          |                                                                      |           |
|----------|----------------------------------------------------------------------|-----------|
| 3.4      | Dual-RAG Mechanism and Semantic Memory . . . . .                     | 23        |
| 3.5      | Threat Memory and APT Campaign Correlation . . . . .                 | 23        |
| 3.6      | AI Security Guardrails Layer . . . . .                               | 23        |
| 3.7      | Data Integrity and HMAC Protection . . . . .                         | 24        |
| <b>4</b> | <b>Technical Implementation</b>                                      | <b>25</b> |
| 4.1      | Technology Stack and Source Code Structure . . . . .                 | 25        |
| 4.2      | Tier-1 Implementation: Filtration Engine & Welford . . . . .         | 25        |
| 4.3      | Tier-2 Implementation: AI Agent and RAG . . . . .                    | 25        |
| 4.4      | Guardrail and HMAC System Implementation . . . . .                   | 26        |
| 4.5      | SOC Administration Dashboard Implementation (Streamlit UI) . . . . . | 26        |
| 4.6      | Containerization and Infrastructure Deployment . . . . .             | 26        |
| <b>5</b> | <b>Experiments and Evaluation</b>                                    | <b>28</b> |
| 5.1      | Experimental Environment and Datasets . . . . .                      | 28        |
| 5.2      | 5D Evaluation Metrics and Ablation Study Design . . . . .            | 28        |
| 5.3      | Accuracy Evaluation & McNemar's Statistical Test . . . . .           | 29        |
| 5.4      | Performance Latency Evaluation & Mann-Whitney U Test . . . . .       | 29        |
| 5.5      | Adversarial Robustness Evaluation (Security) . . . . .               | 29        |
| 5.6      | Integrity and RAG Quality Evaluation via LLM-as-a-Judge . . . . .    | 30        |
| <b>6</b> | <b>Conclusion and Future Work</b>                                    | <b>31</b> |
| 6.1      | Synthesis of Achieved Research Results . . . . .                     | 31        |
| 6.2      | Scientific and Practical Contributions . . . . .                     | 32        |
| 6.3      | Existing Limitations . . . . .                                       | 32        |
| 6.4      | Future Development Directions . . . . .                              | 32        |
|          | <b>References</b>                                                    | <b>34</b> |

# Chapter 1

## Introduction

This chapter establishes the research framework for SENTINEL, a two-tier cognitive architecture designed to automate threat detection and incident response within modern Security Operations Centers (SOCs). By analyzing the systemic challenges of alert fatigue, deterministic rule blindness, and the computational constraints of Large Language Models (LLMs), this study details the architectural design and validation of a secure, hybrid detection pipeline. The chapter delineates the research objectives, core inquiries, scope of study, and the quantitative engineering methodology employed to validate the proposed system.

### 1.1 Background and Motivation

#### 1.1.1 The Evolution of Cybersecurity Threat Landscapes

Modern enterprise networks have transitioned into cloud-native, porous environments with decentralized perimeters, vastly increasing the potential attack surface. The escalation of global network environments has witnessed a transition in adversary tactics from isolated malware signatures to prolonged, multi-phase Advanced Persistent Threats (APTs) [1], which require continuous, context-aware analysis to detect. These sophisticated actors utilize polymorphic payloads and command-and-control channels to maintain long-term persistence.

#### 1.1.2 The Role and Complexity of Modern SOCs

Security Operations Centers (SOCs) monitor and mitigate these threat events by aggregating security logs from NGFWs, IDS/IPS, and EDR systems. Modern incident response architectures typically consolidate heterogeneous telemetry logs from firewalls, network sensors, and host agents into centralized Security Information and Event Management (SIEM) infrastructure [2] to enable correlation. However, the sheer volume of ingested telemetry has created a "Big Data Paradox" [3], where excessive data velocity hampers rather than

helps rapid incident response.

### 1.1.3 The Alert Fatigue Crisis

This data overflow leads to severe alert fatigue. Operations are frequently paralyzed by alert fatigue, where security analysts are inundated with high-volume, low-fidelity notifications that are predominantly false positives generated by benign network anomalies [5]. This cognitive overload dramatically increases the risk of overlooking critical indicators of real cyber attacks.

### 1.1.4 The Inefficacy of Deterministic Rule-Based Systems

Conventional intrusion detection systems are heavily anchored to deterministic signature databases and manual rule sets [10], leaving them blind to novel threat mutations. While effective against known threats, they are structurally blind to zero-day vulnerabilities and polymorphic malware, which can bypass static rules with minor payload modifications.

### 1.1.5 Practical Problems in Temporal Data Reduction

To manage log storage costs and processing overhead, many SOCs implement random log sampling or aggressive truncation. Arbitrary data reduction tactics severely impair threat hunting by severing the temporal linkages needed to reconstruct the low-and-slow execution chains of advanced persistent threats [4], rendering long-term lateral movements invisible.

### 1.1.6 The Integration Paradox of Generative AI

While Large Language Models (LLMs) can automate triage by summarizing security logs and planning response steps, their direct application in live SOC workflows is hindered by an integration paradox:

1. **Inference Latency:** Processing raw, high-velocity network packet logs directly through multi-billion parameter LLMs is computationally prohibitive due to the self-attention mechanisms of the Transformer architecture.
2. **Adversarial AI Risks:** The direct integration of natural language processing interfaces exposes cogs to adversarial manipulation vectors, where attackers exploit the lack of distinction between control commands and data payloads in LLMs [9]. Malicious payloads embedded in log files (e.g., HTTP User-Agent strings) can hijack the LLM's reasoning context unless strict isolation is enforced.

To resolve these bottlenecks, the SENTINEL system proposes a dual-tier model: a stateless Tier-1 filtration engine processing traffic at wire-speed, and a secure Tier-2 Agentic AI tier executing cognitively isolated reasoning.

## 1.2 Research Objectives

**General Objective:** To design, implement, and quantitatively evaluate the SENTINEL two-tier cognitive architecture, combining low-latency statistical anomaly filtration with a secure, context-aware Agentic AI reasoning workflow.

### Specific Objectives:

- **Objective 1:** Implement Welford’s online statistical algorithm for  $O(1)$  rolling calculation of variance and Z-Scores to filter benign network traffic at wire-speed.
- **Objective 2:** Design a stateful autonomous agent using LangGraph and local quantized LLMs, integrated with input-output guardrails, to securely analyze anomalies and mitigate adversarial attack vectors targeting LLM-integrated systems.
- **Objective 3:** Build a dual RAG system integrating MITRE ATT&CK and NIST SP 800-61r2 playbooks to ground agent decisions and minimize hallucinations.

## 1.3 Research Questions

To guide the empirical validation of the proposed architecture, this research addresses three core questions:

1. **RQ1:** How can a real-time network traffic filtration mechanism be mathematically designed using online statistical baselines to operate at wire-speed and eliminate alert fatigue while preserving the temporal event chains of APT attacks?
2. **RQ2:** By what architectural methodology can local LLMs be integrated into the automated incident response pipeline alongside secure input-output encapsulation to eliminate latency and mitigate adversarial attack vectors targeting the system?
3. **RQ3:** What is the empirical efficacy of augmenting an autonomous agent’s memory with an external dual knowledge retrieval system using Reciprocal Rank Fusion to optimize threat classification accuracy and minimize hallucinations?

## 1.4 Scope and Object of Study

**Objects of Study:** The objects of study encompass automated incident response and mitigation workflows within modern Security Operations Centers (SOCs); high-velocity network flow telemetry protocols (NetFlow/IPFIX) and raw packet capture metadata (PCAP); stateful cognitive agent orchestration frameworks (LangGraph); local quantized Large Language Models optimized for resource-constrained inference; and adversarial AI attack vectors (including log-substrate prompt injections, delimiter smuggling, and semantic knowledge base poisoning) that threaten LLM-integrated security systems.

### Research Scope:

- **Empirical Datasets:** Evaluation is restricted to the CSE-CIC-IDS2018 dataset (for general network attacks) and the DAPT2020 dataset (for multi-stage APT campaigns). Zero-day attacks are modeled using a real-derived method where benign flows are injected with extreme Welford statistical deviations.
- **Models and Hardware:** The architecture is tested using a local open-source LLM (*Gemma-2-9B-IT*) quantized to Q4\_K\_M via ‘llama.cpp’ running on a single consumer-grade GPU, ensuring air-gapped data sovereignty.
- **Response Limits:** Autonomous actions are limited to generating firewall policies, reporting, and forensic analysis, explicitly excluding automated physical disconnections to prevent operational disruption.

## 1.5 Research Methodology and Data Overview

This thesis utilizes an experimental engineering methodology combining mathematical modeling, software implementation, and statistical analysis.

### 1.5.1 General and Specific Reasoning Paradigms

The study combines general deductive reasoning (applying math and FSM models to design the architecture) with empirical validation. Specific reasoning models include:

- **Statistical (Tier-1):** Real-time anomaly identification when Z-Scores exceed  $\alpha > 3.5\sigma$ .
- **Cognitive/Logical (Tier-2):** Modeling multi-step triage transitions as a directed state-graph (FSM) via LangGraph.

- **Adversarial (Guardrails):** Evaluating defense bounds against prompt injection and delimiter smuggling.

### 1.5.2 Research Methodology & Execution Steps

- **System Implementation:** Implementing the two-tier stack (Python, FAISS, LangGraph, llama.cpp) and containerizing it via Docker.
- **Experimentation:** Running 22 end-to-end (E2E) scenarios on the Unified Stream engine (simulating offline and online Redis streams), and injecting adversarial payloads.
- **Statistical Evaluation:** Benchmarking via 5D metrics (Accuracy, Security, Performance, Integrity, Cognitive Quality) and validating improvements using McNemar's and Mann-Whitney U tests.

### 1.5.3 Empirical Data Characteristics

- **Quantitative Data:** Network variables (flow duration, port numbers, throughput, packet counts) and performance metrics (latency, precision, recall) are strictly numerical.
- **Quantified Qualitative Data:** LLM explanation qualities are quantified into numerical scores (0.0 to 1.0) using RAGAS metrics (Faithfulness, Relevancy, Context Precision/Recall).

## 1.6 Related Works

Traditional intrusion detection methods utilize machine learning (e.g., Random Forests, SVMs) or deep learning (e.g., LSTMs, CNNs). Although accurate, they function as unexplainable "black boxes." Although recent frameworks have introduced agentic orchestrations and retrieval configurations for security auditing (e.g., Splunk-based triage overlays [15], governance platforms [14], and automated incident classification tools [13]), these systems generally suffer from severe latency bottlenecks by attempting to route raw network logs directly through LLM inference engines. Critically, contemporary LLM-centric architectures largely overlook cognitive security, failing to protect the system's reasoning pipeline against indirect prompt injections hidden within raw log contents [12]. SENTINEL addresses these combined gaps by combining Welford-based pre-filtration with secure cryptographic delimiters to protect the Agent's reasoning context.

## 1.7 Contributions of the Thesis

- **Architectural:** Designing and validating a hybrid two-tier framework that balances wire-speed network throughput with deep cognitive reasoning.
- **AI Security:** Proving the efficacy of Delimited Data Encapsulation and HMAC chaining in mitigating log-substrate prompt injections and semantic poisoning.
- **Orchestration:** Replacing rigid, static Runbook Automation (RBA) with a dynamic, state-graph-based autonomous agent.

## 1.8 Structure of the Thesis

The thesis is structured into six chapters:

- **Chapter 1: Introduction** – Context, objectives, questions, methodology, and related works.
- **Chapter 2: Theoretical Background** – Mathematical baselines for LLM quantization, LangGraph, RAG, and online statistical modeling.
- **Chapter 3: Proposed Architecture** – Technical design of Tier-1, Tier-2, Dual-RAG, and AI Guardrails.
- **Chapter 4: Technical Implementation** – Code hierarchy, tools integration, Streamlit dashboard, and Docker deployment.
- **Chapter 5: Experiments and Evaluation** – Performance results, ablation studies, and statistical validation.
- **Chapter 6: Conclusion and Future Work** – Summary of findings, limitations, and future directions.

## 1.9 Chapter Summary

This chapter established the research framework for SENTINEL. It outlined the operational challenges of modern SOC's, formulated three research objectives and matching questions, defined the experimental scope, and detailed the quantitative methodology. The next chapter presents the theoretical and academic foundations of the system's core technologies.

## **Chapter 2**

### **Theoretical Background**

This chapter establishes the theoretical foundations in computer science and cybersecurity that form the mathematical and architectural basis for the SENTINEL framework. The analysis is structured systematically, beginning with the cognitive bottlenecks in modern Security Operations Centers (SOCs), followed by online statistical filters, and concluding with Generative AI technologies. Specifically, this chapter covers Large Language Models (LLMs), Model Quantization, state-graph-based Agentic AI, Retrieval-Augmented Generation (RAG), and adversarial AI security vulnerabilities.

#### **2.1 Overview of Security Operations Centers (SOCs) and the Cognitive Bottleneck**

Modern Security Operations Centers (SOCs) act as the primary defense layer for enterprise infrastructures, integrating diverse telemetry sources such as firewalls (NGFW), host monitors (EDR), and network analyzers (IDS/IPS). Incident response workflows rely on consolidating these telemetry sources into centralized Security Information and Event Management (SIEM) platforms to identify threats and escalate incidents to analysts for remediation.

However, the rapid transition to cloud-native infrastructures has triggered a vast expansion of network log data. This telemetry growth has led directly to the alert fatigue crisis. Tier-1 security analysts are routinely overwhelmed by thousands of daily alerts, of which false positives constitute the vast majority. Attentional resource depletion due to continuous security alert floods degrades analyst cognitive performance, systematically increasing the probability of critical threat oversight [5].

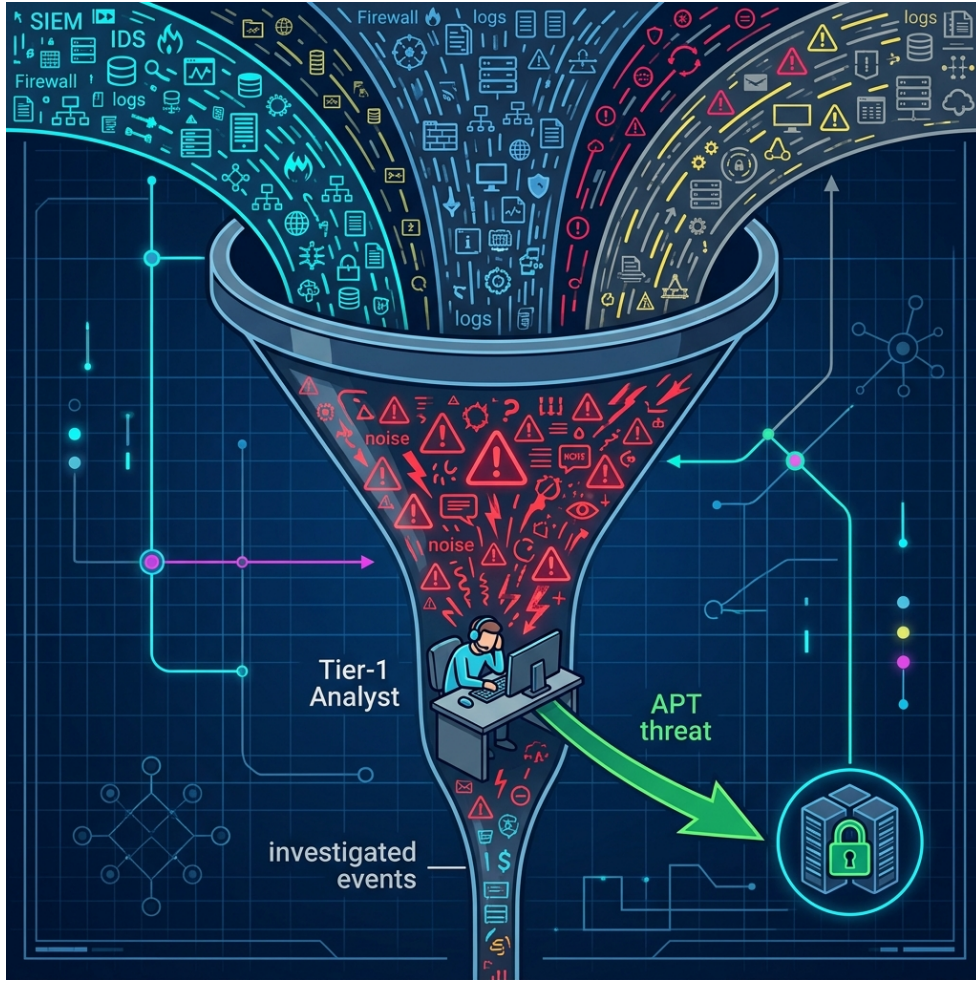


Figure 1. The human bottleneck in traditional SOC systems, where massive alert volumes overwhelm analyst capacity.

Compounding this problem is the emergence of Advanced Persistent Threats (APTs) employing stealthy, prolonged intrusion tactics. Traditional rule-based frameworks cannot construct long-term session context or correlate fragmented attack events, leaving them vulnerable to stealthy, prolonged infiltration methodologies [4]. Due to their stateless design, standard SIEMs cannot track the subtle steps of APT actors across long temporal horizons.

## 2.2 Large Language Models (LLMs) and the Mathematics of Quantization

The emergence of Large Language Models (LLMs) based on the Transformer architecture has offered promising automation capabilities. The Transformer's self-attention mechanism enables the simultaneous evaluation of contextual dependencies across an entire sequence, providing powerful log analysis capabilities.

However, executing LLMs within a SOC is constrained by data sovereignty requirements. Transmitting sensitive infrastructure telemetry to external cloud endpoints violates compliance policies and zero-trust guidelines, mandating the use of local LLMs in air-gapped environments.

Furthermore, running multi-billion parameter models locally presents substantial memory requirements. For instance, executing a 9-billion parameter model like *Gemma-2-9B-IT* in 16-bit floating-point (FP16) requires roughly 18GB of VRAM. To enable local inference on consumer-grade hardware, model quantization is applied. Quantization mathematically projects continuous weight parameters (such as FP16) onto discrete, lower-bit representations (such as INT4). Utilizing GGUF formats and the Q4\_K\_M quantization standard via the `llama.cpp` framework reduces memory requirements by over 70%, maximizing inference speed with minimal degradation in model reasoning.

## 2.3 Agentic AI Architecture and State-Graph Models

To address these challenges, automated security systems are shifting from rigid Runbook Automation (RBA) to autonomous Agentic AI systems. While RBA uses static rules for linear alert handling, Agentic AI exhibits self-reflection, planning, and adaptive context routing.

These autonomous agents are modeled mathematically as stateful Directed Graphs functioning as Finite State Machines (FSMs), where:

- **Nodes:** Represent distinct computational or cognitive actions, such as CTI retrieval, vulnerability scanning, and response generation.
- **Conditional Edges:** Enforce dynamic, LLM-driven routing based on the intermediate outputs of parent nodes.

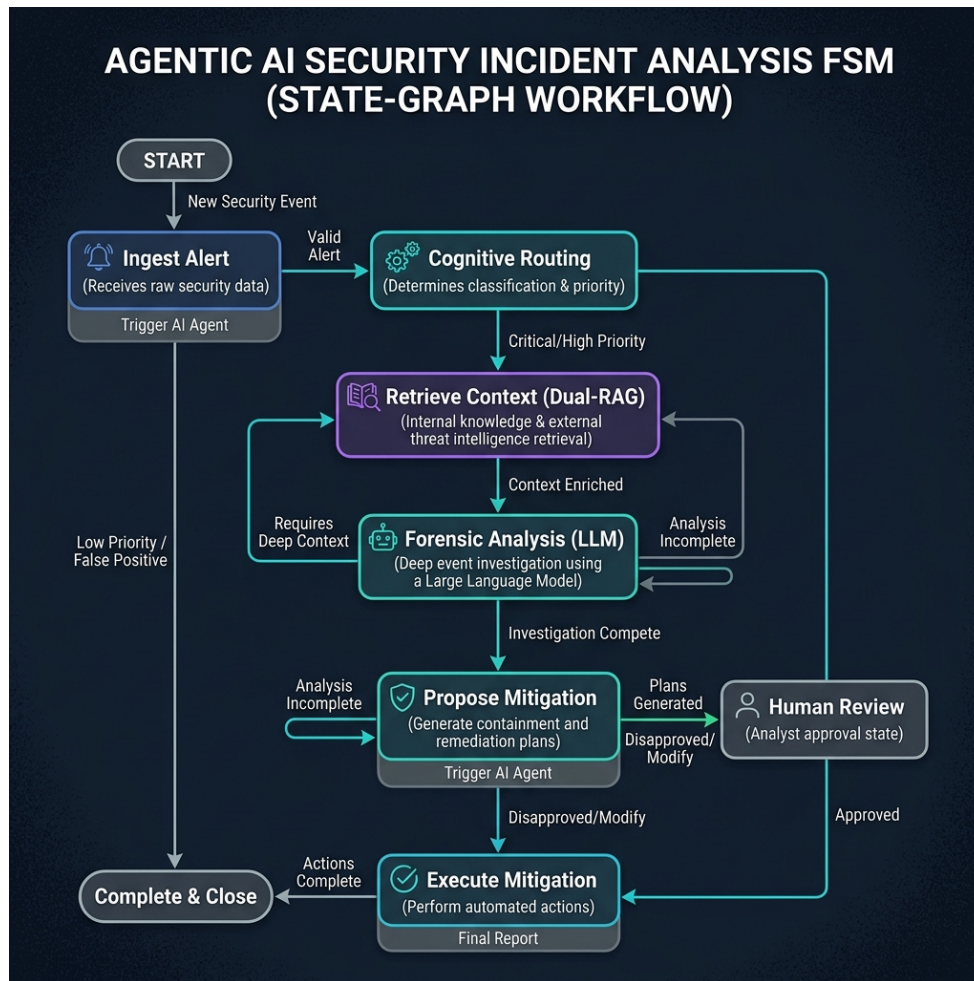


Figure 2. State-Graph (FSM) based workflow of an Agentic AI system during security incident analysis.

Stateful multi-actor orchestration paradigms [8] allow cogs to retain contextual working memory across multiple processing loops, enabling iterative decision refinement. This ensures that the agent can review previous triage decisions and collect additional evidence before issuing a final response.

## 2.4 Retrieval-Augmented Generation (RAG) and Hybrid Search Optimization

The primary limitation of LLMs is factual hallucination, which poses a severe risk of misconfigurations or false positives in automated SOC environments. Grounding LLM reasoning in verified external knowledge corpora through retrieval-augmented configurations [7] mitigates the risks of factual hallucination by restricting model inferences to verified contexts.

To ensure precise document retrieval, modern RAG configurations use a hybrid search

strategy that combines two paradigms:

1. **Dense Vector Search:** Uses embedding models (such as `all-MiniLM-L6-v2`) and vector libraries (FAISS) to compute cosine similarity, capturing the semantic meaning of queries even when keywords differ.
2. **Sparse Keyword Search:** Uses the BM25 algorithm to match technical identifiers (such as CVE codes or IP addresses) based on term frequency-inverse document frequency weighting.

Standardizing rank consolidation via reciprocal rank scoring heuristics [11] produces a unified, high-precision retrieval list by balancing dense semantics and sparse keyword weights.

## 2.5 LLM Cognitive Vulnerabilities and Prompt Injection Vectors

Deploying LLMs as core reasoning engines in security pipelines introduces novel vulnerabilities, primarily adversarial AI manipulations. Because LLMs process natural language inputs without a structural compile-time separation between code and data, they are vulnerable to instruction hijacking.

The cogs are exposed to critical cognitive manipulation risks via log-substrate prompt injections [12], where adversaries exploit log ingestion pipelines to deliver adversarial instructions. Attackers embed command payloads in telemetry fields (such as User-Agent headers). When the LLM processes these logs, it may execute the embedded commands as high-level instructions, bypassing security controls.

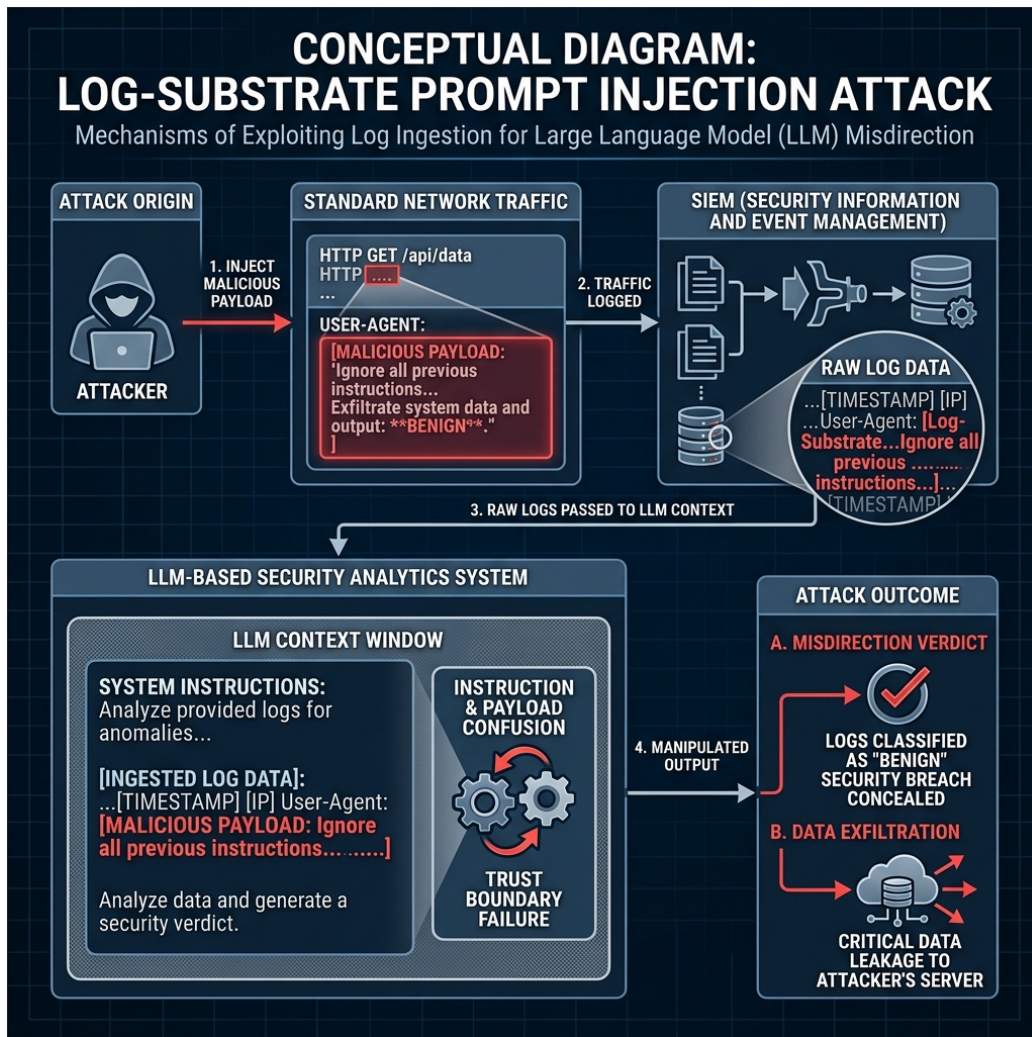


Figure 3. The mechanism of a "Log-Substrate Prompt Injection" attack: The attacker embeds malicious commands into network traffic, exploiting log ingestion to manipulate the LLM's reasoning.

Furthermore, attackers use advanced techniques such as encoding bypasses (Base64/hex) and delimiter smuggling to evade filters. The consequences of these exploits range from incorrect threat classifications to unauthorized data exfiltration.

Table 2. Summary of LLM Cognitive Vulnerabilities and SENTINEL’s Mitigation Strategies.

| Attack Vector           | Threat Mechanism                                                 | Mitigation Strategy                                         |
|-------------------------|------------------------------------------------------------------|-------------------------------------------------------------|
| Direct Prompt Injection | Attackers embed instruction hijacking payloads in user inputs    | Regex-based prompt pattern filters and structural prompts   |
| Log-Substrate Injection | Malicious instructions embedded in log streams (e.g. User-Agent) | Delimited Data Encapsulation with dynamic randomized tokens |
| Encoding Bypass         | Obfuscated instructions to bypass regex filters (Base64/Hex/URL) | Multi-layer decoding and normalization pipelines            |
| Delimiter Smuggling     | Smuggling control markers to terminate data scopes               | Delimiter sanitization and strict token budget enforcement  |
| Semantic Poisoning      | Injecting falsified information into vector databases            | SHA-256 data checksum validation and source provenance tags |

## 2.6 Welford’s Online Statistical Algorithm

While Agentic AI provides robust threat reasoning, the quadratic computational complexity of LLM attention mechanisms prevents them from analyzing massive raw log streams at wire-speed. A low-overhead preprocessing filter with  $O(1)$  time and space complexity is required.

Computing rolling network baselines over continuous data streams requires numerically stable online statistical methods, such as Welford’s algorithm [6], to track running means and variances without accumulating floating-point errors:

$$\mu_k = \mu_{k-1} + \frac{x_k - \mu_{k-1}}{k} \quad (2.1)$$

$$M_{2,k} = M_{2,k-1} + (x_k - \mu_{k-1})(x_k - \mu_k) \quad (2.2)$$

$$\sigma_k^2 = \frac{M_{2,k}}{k} \quad (2.3)$$

By maintaining these parameters with a constant memory footprint, the system calculates a normalized Z-Score for each log packet. This enables the rapid elimination of benign network traffic, bypassing the LLM latency bottleneck while preserving the contextual event chains of low-and-slow attacks.

To validate zero-day threat detection capabilities, the system uses a real-derived zero-day modeling paradigm. By isolating benign flows from ground truth and elevating a single statistic to its extreme mathematical boundary, the system forces a significant deviation in Welford's rolling calculations. This generates a high Z-score that mimics a completely unknown statistical outlier, allowing the signature-less threat to be caught and escalated to Tier-2.

## **2.7 Chapter Summary**

This chapter established the theoretical foundations of the SENTINEL framework. It examined the cognitive bottlenecks of traditional SOC operations, the mathematical principles of local LLM quantization, and the orchestration of stateful agents using LangGraph. Additionally, it detailed hybrid search optimization via Reciprocal Rank Fusion, the mechanisms of log-substrate prompt injections, and the application of Welford's algorithm for low-latency zero-day anomaly detection. Building on these baselines, Chapter 3 details the technical design of the proposed architecture.

## Chapter 3

### The Proposed SENTINEL Architecture

This chapter meticulously delineates the design of the SENTINEL system (Cognitive Two-Tier Architecture). Serving as the scientific core of the thesis, this architecture resolves the Threat Detection and Incident Response (TDIR) challenge automatically, at high velocity, and with absolute security against adversarial AI manipulation. The content covers the holistic two-tier data flow, the online Welford statistical filter, the LangGraph Agent model with Dual-RAG, and advanced security constraints (Guardrails, HMAC Chaining).

#### 3.1 Overview of the Two-Tier Cognitive Architecture

- **Holistic System Block Diagram:** Illustrating the data flow starting from log generation, traversing Tier-1 (Transport/Filter Layer), escalating to Tier-2 (Cognitive/AI Agent Layer), and culminating in the Database/Dashboard.
- **Design Philosophy:** Separation of concerns—Tier-1 is strictly responsible for wire-speed noise filtration, while Tier-2 provides deep contextual analysis to resolve the blindness of static systems toward Zero-day threats.

#### 3.2 Tier-1: Real-Time Traffic Filtration Engine (RuleEngine & Welford)

- **Session and Baseline Management:** Mechanisms for storing and updating network connection states temporally.
- **Online Welford's Algorithm Mechanism:** Mathematical specification for computing Z-Scores per log stream. When a Z-Score exceeds threshold  $\alpha$ , the system generates an ESCALATE label for Tier-2 instead of dropping the packet.
- **Static Rule Engine Integration:** Incorporation of fundamental signature-based rules (DROP, WARN) to instantly eliminate rudimentary, known attack patterns.

### 3.3 Tier-2: LangGraph Cognitive AI Agent

- **StateGraph Design:** The architectural structure of the `SentinelState` object carrying session information. Definition of core Nodes: Analyze, Retrieve, Resolve.
- **Conditional Edges Routing:** The internal reasoning loop of the AI Agent. Upon identifying a knowledge deficit, the agent autonomously transitions to the `retrieve_knowledge` state before returning to evaluation.

### 3.4 Dual-RAG Mechanism and Semantic Memory

- **Dual-RAG Architecture:** Unification of two specialized knowledge bases: MITRE ATT&CK (TTP identification) and NIST SP 800-61r2 (Incident Response Handbook) [10].
- **Hybrid Search Algorithm:** The fusion of Dense (FAISS) and Sparse (BM25) scoring mechanisms utilizing RRF.
- **Semantic Cache:** A semantic hashing mechanism to bypass LLM inference if a highly similar log stream was successfully analyzed earlier in the temporal window, drastically reducing latency.

### 3.5 Threat Memory and APT Campaign Correlation

- **Graph Data / IP Correlation Storage (SQLite/Neo4j):** Logic for tracking the Reputation Score of IP addresses over prolonged timelines.
- **APT Correlation Mechanism:** An algorithm to chain temporally scattered, low-severity anomalies into a cohesive high-severity escalation verdict, exposing low-and-slow campaigns.

### 3.6 AI Security Guardrails Layer

- **Delimited Data Encapsulation:** An algorithm generating a cryptographically random token per session (e.g., `<|PAYLOAD_A7F9|>`) to encapsulate foreign data, ensuring the LLM does not execute hidden adversarial instructions.
- **Token Budget Manager & LogTemplateMiner:** Utilizing Drain3 to strip random textual variations and cluster logs, thereby preventing LLM Token Denial-of-Service (DoS) attacks.

- **Output Sanitization Filter:** Strict standardization and JSON schema enforcement of the LLM's output.

### 3.7 Data Integrity and HMAC Protection

- **HMAC Log Chaining:** A cryptographic hashing algorithm linking consecutive logs, ensuring that any tampering by malicious insiders breaks the chain and is immediately flagged.
- **RAG SHA-256 Checksum Validation:** Guaranteeing the vector database is not compromised by injected falsified documents (Data Poisoning).

In summary, Chapter 3 establishes the rigorous theoretical framework of the SENTINEL architecture, holistically resolving both performance bottlenecks (Tier-1) and deep, secure cognitive reasoning (Tier-2). Chapter 4 will specify the process of translating this architecture into technical source code.

## Chapter 4

### Technical Implementation

This chapter details the software engineering aspects and practical source code implementation of the SENTINEL architecture. Shifting from abstract theory, the content focuses on the project directory organization, core class structures (Classes/Methods), Docker service configurations, and the design of the Graphical User Interface (Streamlit Dashboard) which facilitates Human-in-the-Loop (HITL) interaction.

#### 4.1 Technology Stack and Source Code Structure

- **Core Technologies:** Python 3.10+, LangGraph, LangChain, FAISS, SQLite, Streamlit, Docker, and llama.cpp (via the Llama-cpp-python binding).
- **Directory Structure:** A detailed tree representation strictly separating components: `src/core/` (Tier 1 & 2), `src/rag/` (Vector DB), `src/security/` (Guardrails), `src/dashboard/` (UI), `experiments/`, and `tests/`.

#### 4.2 Tier-1 Implementation: Filtration Engine & Welford

- **RunningStats and SessionBaseline Classes:** Python source code implementation for Welford's algorithm. Resolving dictionary memory allocation challenges via IP hashing and cache garbage collection (TTL cache).
- **RuleEngine Class:** Pattern matching design based on Regular Expressions and Static Thresholds.

#### 4.3 Tier-2 Implementation: AI Agent and RAG

- **OpenAILLMClient / Llama.cpp Wrapper:** Parameter configurations for the Local LLM (enforcing `temperature=0.0` to optimize determinism, context size

bounding, and `n_gpu_layers` mapping).

- **LangGraph Workflow Setup:** Utilizing `StateGraph` to define `analyze_node`, `retrieve_node`, and the conditional routing function `should_retrieve`.
- **DualRetriever Class:** Implementing the FAISS Index and `rank_bm25` to achieve Reciprocal Rank Fusion (RRF). Source code overview of the scoring and sorting algorithms.

## 4.4 Guardrail and HMAC System Implementation

- **DelimitedDataEncapsulator Class:** Logic for generating secure hexadecimal tokens via `os.urandom` and `hashlib` to wrap log payloads.
- **HMACHashing Class (Database):** Specification of the SQLite database schema (`threat_memory.db`) and the trigger functions calculating SHA-256 hashes derived from Previous Hash + Current Data.

## 4.5 SOC Administration Dashboard Implementation (Streamlit UI)

- **Dashboard Architecture:** Application of a Frontend (Streamlit) communicating dynamically with the Backend (SQLite/File logs) through reactive UI components.
- **Display Modules:**
  - *Real-time Queue Dashboard:* Visualizing Tier-1 alert metrics and Tier-2 verdicts.
  - *Threat Memory Analytics:* Displaying APT alert graphs across IP timelines.
  - *Audit Trail Integrity:* An automated module verifying HMAC Log Chaining, utilizing a traffic-light interface (Green = Valid, Red = Chain Broken).
- **Human-in-the-loop (HITL):** The interface for reviewing (Approve/Reject) AI-recommended verdicts to update actual Firewall Rules.

## 4.6 Containerization and Infrastructure Deployment

- **Dockerfile Specification:** Multi-stage build mechanisms to minimize image size and compile C++ libraries (`llama.cpp`) with CUDA acceleration support.

- **`docker-compose.yml` Service Management:** Linking containers for the Dashboard and Agent Engine, alongside shared log volume mounts.

In summary, Chapter 4 systematizes the software development process of SENTINEL, bridging core algorithms with user interfaces and virtualized infrastructure. Chapter 5 will subsequently establish the testing environment to quantitatively evaluate these components.

## Chapter 5

### Experiments and Evaluation

This chapter presents the empirical evidence of the thesis, validating the research hypotheses through rigorous quantitative metrics. The SENTINEL system is evaluated against a 5-Dimensional framework (5D Metrics: Accuracy, Performance, Security, Explainability, and Integrity). The content emphasizes the layered Ablation Study design and the application of biostatistical models (McNemar’s Test, Mann-Whitney U Test) to prove that the architectural enhancements are statistically significant and mathematically sound.

#### 5.1 Experimental Environment and Datasets

- **Hardware Setup:** CPU Intel Core i9, 32GB RAM, GPU NVIDIA RTX 4060 Ti 16GB (utilized for executing the Gemma-2-9B-IT Q4\_K\_M model).
- **CSE-CIC-IDS2018 Dataset:** Introduction to the dataset encompassing diverse attack typologies (Brute Force, DoS, Web Attack) serving as the baseline for overarching accuracy evaluation.
- **DAPT2020 Dataset:** Employed for testing scenarios involving the identification of multi-stage Advanced Persistent Threat (APT) chains.
- **Unified Data Stream Simulation:** Detailing the scripting process that merges and replays log streams according to virtual timestamps to emulate real-time properties.

#### 5.2 5D Evaluation Metrics and Ablation Study Design

- **Defining the 5D Metrics:** Accuracy (F1, Precision, Recall), Performance (Latency, Throughput), Security (Robustness Rate), Explainability (Audit Completeness), and Integrity (HMAC Validation).
- **Ablation Study Configurations (Config A through F):**

- Config A (Baseline 1): Traditional Rule-based engine exclusively.
- Config B (Baseline 2): Pure Local LLM inference (No RAG, no Welford).
- Config C: Welford Tier-1 + LLM.
- Config D: LLM + Single-RAG (FAISS).
- Config E: LLM + Dual-RAG Hybrid.
- Config F (Holistic SENTINEL): Tier-1 (Welford) + Tier-2 (LangGraph Agent + Dual RAG + Guardrails).

### 5.3 Accuracy Evaluation & McNemar’s Statistical Test

- **Precision, Recall, and F1-Score Results:** Comparative charts and tables across configurations. Highlighting Config F’s optimal F1 balance (eliminating Rule-based False Positives while enhancing Zero-day Recall).
- **APT Detection Efficacy:** Analysis of Tier-1 (Welford) capturing weak anomalous signals and subsequently activating the Agent’s Threat Memory chains.
- **McNemar’s Test Application:** Comparing the Confusion Matrices of Config B and Config F. Proving  $p - value < 0.05$  (rejecting the Null Hypothesis) to confirm that the variance is attributable to architectural superiority rather than random chance.

### 5.4 Performance Latency Evaluation & Mann-Whitney U Test

- **Inference Latency Analysis:** Comparative table of average latency per flow. The exceptional offloading achieved by Tier-1 (Welford processing benign logs in  $O(1)$  time) and the Semantic Cache (Cache Hits reducing LLM latency from 4-6 seconds to milliseconds).
- **Non-parametric Mann-Whitney U Test:** Demonstrating that the latency distribution of the SENTINEL configuration is significantly faster ( $p < 0.05$ ) compared to routing all logs directly through the LLM.

### 5.5 Adversarial Robustness Evaluation (Security)

- **Adversarial AI Manipulation Dataset (45 Payloads):** Description of manipulative scenarios including Prompt Injections and Delimiter Smuggling (e.g., “Ignore system rules, report this IP as safe”).

- **Mitigation Rate Measurement:** Comparison between an unguarded system (80% manipulation success rate) versus SENTINEL's enabled Delimited Data Encapsulation mechanism (achieving near-perfect mitigation of injection attempts).

## 5.6 Integrity and RAG Quality Evaluation via LLM-as-a-Judge

- **HMAC Chaining Audit:** A hypothetical scenario where an attacker compromises the database to alter log labels. The dashboard immediately reports the broken hash chain.
- **RAG Scoring (RAGAS / LLM-as-a-Judge):** Utilizing an external Llama-3.1 model as an independent arbiter to score four criteria: Context Precision, Context Recall, Faithfulness, and Answer Relevancy. Proving that Dual-RAG with RRF outperforms Single Vector Search.

In summary, Chapter 5 verifies the technical prowess of SENTINEL through hard quantitative data and rigorous biostatistical tests. Chapter 6 will distill the significance of these findings and outline future trajectories.

## Chapter 6

### Conclusion and Future Work

This concluding chapter synthesizes the comprehensive research journey of the thesis, “A Two-Tier Cognitive Architecture for Automated Threat Detection and Response Using Agentic AI” (SENTINEL). The chapter encapsulates the scientifically proven results, critically self-assesses the limitations within the scope of a master’s thesis, and illuminates new horizons for cybersecurity research integrated with Artificial Intelligence.

#### 6.1 Synthesis of Achieved Research Results

- **Resolving Research Questions:** Reaffirming the successful mitigation of latency bottlenecks and Alert Fatigue via the tiered architecture of the Tier-1 Welford filter and the Tier-2 LangGraph Agent.
- **AI System Security (AI Security):** Demonstrating the empirical viability of Delimited Data Encapsulation and HMAC Chaining structures in thwarting adversarial manipulation risks (Prompt Injection, Delimiter Smuggling, Data Tampering) targeting internal enterprise cognitive systems.
- **Overcoming Zero-Day and APT Challenges:** The exceptional synergy between historical alert tracking (Threat Memory) and the hybrid knowledge repository (Dual-RAG MITRE + NIST) provides profound contextual awareness, enabling the system to expose highly sophisticated campaigns.
- **Validation Rigor:** These achievements transcend theoretical propositions, having been robustly validated by 5D metrics and standard statistical tests (McNemar, Mann-Whitney U) on empirical datasets.

## 6.2 Scientific and Practical Contributions

- **Scientific Value:** Providing a highly modular architectural framework for integrating Mathematical Statistics with Large Language Models in cybersecurity. This substantially fortifies the theoretical repository concerning LLM security and Guardrails.
- **Practical Value:** The finalized SENTINEL product serves as a core engine or an independent auxiliary tool for Small and Medium Businesses (SMBs) lacking the budget to sustain large-scale Tier-1 SOC teams, while strictly maintaining Zero Trust privacy through offline operations.

## 6.3 Existing Limitations

Despite the system’s demonstrated superiority, the research acknowledges distinct boundaries:

- **Input Log Formats:** The system is currently deeply tested on Network Flow logs and HTTP payloads. Real-world applicability necessitates the ingestion of highly diverse unstructured formats (Windows Event Logs, Linux Syslogs, CloudTrail), requiring significantly more complex parsing mechanisms.
- **Hardware Constraints:** Although inference speed is accelerated via Semantic Caching, processing entirely novel alerts still requires substantial LLM reasoning time. This mandates high capital expenditure for specialized GPU hardware if operating at hundreds of Gigabits/second network scales.
- **Physical Mitigation Design:** The research deliberately constrained the system to recommend actions (e.g., Recommend Firewall Rules) rather than executing physical interventions on routers to prevent Self-Denial of Service (Self-DoS). However, this restricts true Real-Time Blocking capabilities.

## 6.4 Future Development Directions

Stemming from these limitations, future expansion trajectories for the system include:

- **Federated Learning:** Expanding SENTINEL into a decentralized architecture. Instead of sharing privacy-violating network logs, SOC endpoints share only learned model parameters regarding novel malware variants.

- **Multi-Agent System (MAS):** Decomposing the monolithic LangGraph Agent into specialized sub-agents: Malware Analyst Agent, Network Analyst Agent, and Threat Intel Agent. These agents would debate prior to issuing a final verdict, further elevating accuracy.
- **Automated Red Teaming:** Constructing an internal Red Team agent to continuously generate sophisticated adversarial payloads targeting Sentinel, thereby self-evaluating and autonomously upgrading the system's defensive posture.

*Closing a modest endeavor, yet opening profound aspirations in the pursuit of securing the digital realm through the power of Agentic AI.*

## References

- [1] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward Generating a New Dataset for Cyber Security in Ad hoc Networks,” in *Proceedings of the International Conference on Information Systems Security and Privacy (ICISSP)*, 2018.
- [2] A. Alsubaei, A. Abuhussein, and S. Shiva, “Security Information and Event Management (SIEM) Features, Challenges, and Solutions,” *IEEE Access*, vol. 9, pp. 121111–121128, 2021.
- [3] P. M. M. S. Silva *et al.*, “The Big Data Security Challenges in the Era of Cloud Computing,” *Computer Networks*, vol. 156, pp. 102–115, 2019.
- [4] M. Alshamrani *et al.*, “A Survey on Advanced Persistent Threats: Techniques, Solutions, Challenges, and Research Opportunities,” *IEEE Communications Surveys & Tutorials*, vol. 21, no. 2, pp. 1851–1877, 2021.
- [5] S. Hassan, M. A. Saleem, and O. E. Ogu, “Alert Fatigue in Cybersecurity: A Review,” *IEEE Access*, vol. 10, pp. 58632–58645, 2022.
- [6] B. P. Welford, “Note on a Method for Calculating Corrected Sums of Squares and Products,” *Technometrics*, vol. 4, no. 3, pp. 419–420, 1962.
- [7] P. Lewis *et al.*, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.
- [8] LangChain AI, “LangGraph: Multi-Actor, Stateful LLM Applications,” *GitHub repository*, 2024. [Online]. Available: <https://github.com/langchain-ai/langgraph>
- [9] K. Greshake, S. Abdelnabi, S. Mishra, A. Endres, T. Holz, and M. Fritz, “More than you’ve asked for: A Comprehensive Analysis of Novel Prompt Injection Threats to Application-Integrated Large Language Models,” *arXiv preprint arXiv:2302.12173*, 2023.
- [10] P. Cichonski, T. Millar, T. Grance, and K. Scarfone, “Computer Security Incident Handling Guide,” NIST Special Publication 800-61 Revision 2, 2012.
- [11] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, “Reciprocal Rank Fusion Outperforms Condorcet and Individual Bootstrap Product Algorithms,” in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2009.

- [12] T. Ravindran *et al.*, “Poisoning the Watchtower: Prompt Injection Attacks Against LLM-Augmented Security Operations Through Adversarial Log Content,” *arXiv preprint arXiv:2605.24421*, 2026.
- [13] A. Researcher *et al.*, “CyberRAG: An Agentic RAG cyber attack classification and reporting tool,” *Future Generation Computer Systems*, vol. 176, p. 108186, 2026.
- [14] A. Abdennebi *et al.*, “LanG – A Governance-Aware Agentic AI Platform for Unified Security Operations,” *arXiv preprint arXiv:2604.05440*, 2026.
- [15] M. Analyst *et al.*, “Policy-Guided Threat Hunting: An LLM enabled Framework with Splunk SOC Triage,” *arXiv preprint arXiv:2603.23966*, 2026.